



**Universidad
Europea**

UNIVERSIDAD EUROPEA DE MADRID

**ESCUELA DE ARQUITECTURA, INGENIERÍA Y
DISEÑO**

GRADO EN FÍSICA

TRABAJO FIN DE GRADO

**APLICACIÓN DE TENSOR
NETWORKS EN MODELOS DE IA
PARA CLASIFICACIÓN DE IMÁGENES**

MANUEL ARENAS SÁNCHEZ

Dirigido por

ALEJANDRO MATA ALI

MARÍA FUENCISLA GILSANZ MUÑOZ

CURSO 2025-2026

TÍTULO: APLICACIÓN DE TENSOR NETWORKS EN MODELOS DE IA PARA CLASIFICACIÓN DE IMÁGENES

AUTOR: MANUEL ARENAS SÁNCHEZ

TITULACIÓN: GRADO EN FÍSICA

DIRECTOR/ES DEL PROYECTO: ALEJANDRO MATA ALI
MARÍA FUENCISLA GILSANZ MUÑOZ

FECHA: MARZO DE 2026

RESUMEN

ESTO SE ESCRIBE AL FINAL DEL PROYECTO

ABSTRACT

Keywords: Palabras clave en inglés

AGRADECIMIENTOS

Se escribe al final del proyecto

Cita - frase célebre / Dedicatoria

Escribiré algo al final del proyecto

TABLA RESUMEN

[Rellena esta tabla, y borra la opción que no aplica. Si tu proyecto ha tenido tanto desarrollo de producto como investigación, haz constar “SI” en ambos casos]

	DATOS
Nombre y apellidos:	Manuel Arenas Sánchez
Título del proyecto:	Aplicacion de Tensor Networks en modelos de IA para clasificación de imágenes...
Directores del proyecto:	Alejandro Mata Ali y Maria Fuencisla Gilsanz
El proyecto se ha realizado en colaboración de una empresa o a petición de una empresa:	SI
El proyecto ha implementado un producto: (esta entrada se puede marcar junto a la siguiente)	NO
El proyecto ha consistido en el desarrollo de una investigación o innovación: (esta entrada se puede marcar junto a la anterior)	SI
Objetivo general del proyecto:	[... una frase...]

ÍNDICE

RESUMEN	4
ABSTRACT	5
AGRADECIMIENTOS	6
TABLA RESUMEN	8
1 INTRODUCCIÓN	14
1.1 Estado del Arte	14
1.2 Planteamiento del Problema	16
1.3 Planificación del TFG	18
1.4 Recursos requeridos	20
2 OBJETIVOS	21
2.1 Objetivos generales	21
2.2 Objetivos específicos	21
2.3 Objetivos que quedan fuera del proyecto	22
2.4 Beneficios del proyecto	22
3 MARCO TEÓRICO	23
3.1 Deep learning y clasificación de imágenes	23
3.2 Tensores	25
3.3 Notación diagramática de Penrose y definición de Tensor Network	27
3.4 Tensor Networks y reducción de dimensionalidad	29
3.5 Descomposiciones tensoriales fundamentales	30
3.6 Topologías principales de Tensor Networks	32
3.6.1 Matrix Product State / Tensor Train	32
3.6.2 Tree Tensor Network	33

3.6.3	MERA	34
3.6.4	PEPS	34
4	METODOLOGÍA	36
4.1	Pipeline computacional de clasificación en PyTorch	36
4.2	Organización del proyecto	38
4.3	Dataset utilizado	39
4.4	El papel del embedding	40
4.5	Construcción del modelo de TN	41
4.6	Cambio de embedding	43
4.7	Entrenamiento, regularización y métricas de estudio	44
4.8	Evolución experimental del proyecto	46
4.9	Ejecución en servidores y registro experimental	46
5	RESULTADOS	48
6	DISCUSIÓN	49
7	CONCLUSIONES	50
7.1	Conclusiones del trabajo	50
7.2	Conclusiones personales	50
	REFERENCIAS	52
	ANEXOS	53

ÍNDICE DE FIGURAS

1.1	Planificación por semanas del TFG, cada circulo representa una semana teniendo febrero 3 semanas (porque empiezo el 9 de Febrero), marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.	18
3.1	Esquema de una red neuronal simple. [1]	23
3.2	Esquema de topologías básicas de redes neuronales, desde el perceptrón hasta arquitecturas con capas ocultas y conexiones recurrentes [2].	25
3.3	Representación de una TN inventada. En esta representación se pueden identificar muchas de las operaciones descritas en esta sección. Elaboración propia.	28
3.4	Ejemplo de representación diagramática para un tensor. Fuente: [3]	29
3.5	Diagramas TN de algunas descomposiciones TN populares. (a) El formato CP descompone un tensor X en una suma de varios tensores de rango 1 $a_{:,r}^{(1)} \circ a_{:,r}^{(2)} \circ \dots \circ a_{:,r}^{(N)}$. (b) La descomposición de Tucker descompone un tensor X en un tensor núcleo G multiplicado por una matriz $A^{(n)}$ a lo largo del modo n -ésimo. (c) La descomposición en términos de bloque descompone un tensor X en una suma de varias descomposiciones de Tucker, con rangos de Tucker bajos. (d) La descomposición TT descompone un tensor X en una multiplicación lineal de un conjunto de tensores núcleo de tercer orden $G^{(2)} \dots G^{(N-1)}$ y dos matrices $G^{(1)}$ y $G^{(N)}$. (e) La descomposición TR descompone un tensor X en un conjunto de tensores núcleo de tercer orden y los contrae en una estructura de anillo. (f) La descomposición HT representa un tensor X como un diagrama con estructura de árbol. (g) La descomposición en malla tensorial, también conocida como PEPS, representa un tensor de alta dimensión como una malla bidimensional de tensores interconectados de menor rango, donde cada nodo se conecta con sus vecinos para capturar eficientemente correlaciones espaciales en sistemas con interacciones locales. [4]	32
4.1	Pipeline completo del modelo propuesto	38
4.2	Matriz de confusion obtenida experimentalmente utilizando el modelo <i>HybridCustomTTN</i>	45

4.3	Captura de pantalla de una GPU ADA 4000 del servidor Lorca de la UEM ejecutando dos procesos de entrenamientos diferentes en paralelo.	47
-----	---	----

ÍNDICE DE TABLAS

Todas las tablas incluidas en la memoria deben estar referenciadas para incluir en este índice.

3.1	Comparación entre las descomposiciones CP, Tucker y TT/MPS.	31
3.2	Comparativa sintética de topologías relevantes de Tensor Networks.	35

Capítulo 1. INTRODUCCIÓN

La clasificación de imágenes es una de las tareas centrales del *machine learning*. Consiste en asignar a una imagen una etiqueta entre un conjunto finito de clases, a partir de patrones visuales que pueden depender, de forma simultánea, del color, la forma, la textura, la escala, la iluminación y el contexto. En los últimos años, los modelos de *deep learning* han demostrado una gran capacidad para resolver este tipo de problemas, pero también han puesto de manifiesto un problema importante: los modelos más expresivos suelen requerir un número elevado de parámetros, grandes cantidades de datos y un coste computacional considerable.

Este TFG parte de esa problemática y estudia una familia de modelos alternativa, inspirada en herramientas procedentes de la física cuántica y del álgebra multilineal: las Tensor Networks (TN). La idea general es representar objetos de dimensión muy alta mediante la contracción de tensores más pequeños, de forma que el número de parámetros crezca de manera controlada.

La motivación física aparece de manera natural. En mecánica cuántica, describir exactamente un sistema de muchas partículas exige manejar espacios de Hilbert cuya dimensión crece exponencialmente con el número de componentes. Las TN surgieron precisamente de una idea que sirvió para representar de forma eficiente estados cuánticos relevantes sin almacenar explícitamente todos sus coeficientes. En *machine learning* aparece un problema análogo; una imagen contiene muchas variables, y una función de decisión general sobre todas ellas vive, en principio, en un espacio de dimensión enorme. La pregunta que guía este trabajo es, si las estructuras tensoriales pueden aprovechar regularidades de los datos visuales, para construir clasificadores entrenables y analizables.

1.1 Estado del Arte

La literatura reciente sobre las Tensor Networks aplicadas a *machine learning* puede organizarse en tres líneas principales. La primera utiliza descomposiciones tensoriales para reducir dimensionalidad o comprimir modelos ya existentes. En esta línea, las TN se emplean como herramienta de factorización de pesos, capas densas, convoluciones o representaciones intermedias, con el objetivo de reducir memoria y coste computacional [5, 4]. La segunda línea utiliza las TN como modelos de aprendizaje en sí mismos, es decir, como clasificadores o generadores entrenables. La tercera, más reciente, estudia arquitecturas híbridas que combinan ideas tensoriales con mecanismos habituales de *deep learning*, como normalización, *patch embeddings*, capas convolucionales ligeras o entrenamiento extremo a extremo con *backpropagation* [4].

Un punto de partida fundamental para la aplicación de TN a clasificación supervisada

es el trabajo de Stoudenmire y Schwab [6]. En dicho trabajo, una imagen se transforma mediante un mapa local de características y el clasificador se interpreta como una función lineal en un espacio de características de dimensión exponencial, cuyo tensor de pesos se representa de forma compacta mediante una Matrix Product State (MPS). El resultado mostró que una MPS podía alcanzar un error de test inferior al 1% en MNIST. Este resultado fue importante porque demostró que las redes tensoriales podían actuar como modelos supervisados, no solo como herramientas en física cuántica o como herramientas de compresión. Sin embargo, también marcó una limitación que sigue siendo relevante: la MPS impone una estructura lineal sobre los datos. En imágenes sencillas y pequeñas, como MNIST, esta linealización puede ser suficiente; en imágenes naturales, en color o de mayor resolución, la pérdida de localidad espacial se vuelve mucho más problemática.

Una respuesta natural a esa limitación consiste en utilizar topologías que respeten mejor la estructura bidimensional de las imágenes. Los Projected Entangled Pair States (PEPS) son especialmente atractivos porque organizan los tensores sobre una rejilla 2D, más cercana a la geometría original de una imagen. Cheng, Wang y Zhang aplicaron PEPS a clasificación supervisada en MNIST y Fashion-MNIST, mostrando que una red tensorial bidimensional puede superar a topologías tipo árbol en esos escenarios y funcionar con un número reducido de parámetros [7]. No obstante, PEPS introduce una dificultad importante: la contracción exacta de redes tensoriales bidimensionales es computacionalmente intratable en el caso general [8]. Por ello, aunque PEPS es conceptualmente muy adecuado para imágenes, su uso práctico suele requerir contracciones aproximadas, restricciones en las dimensiones de enlace o implementaciones cuidadosas.

Entre la MPS lineal y PEPS bidimensional aparecen las Tree Tensor Networks (TTN), que organizan la información de manera jerárquica. Esta familia resulta interesante porque reduce la distancia efectiva entre regiones alejadas de la entrada y permite agregar información de forma progresiva. En comparación con una cadena MPS, una TTN puede capturar interacciones de mayor alcance con menos pasos de propagación. Aun así, al no tener ciclos, las interacciones entre ramas distintas quedan mediadas por ancestros comunes, lo que limita la comunicación directa entre regiones separadas. Por tanto, la TTN representa un compromiso entre eficiencia de contracción, estructura jerárquica y capacidad expresiva [4, 9].

La tendencia más reciente no se limita a elegir una topología física clásica, sino a reinterpretar las TN desde el punto de vista de las arquitecturas modernas de visión. Un ejemplo especialmente relevante es Deep Tree Tensor Networks (DTTN), que propone apilar módulos de interacción antisimétrica (AIM) para capturar interacciones multiplicativas de orden creciente y mantener, al mismo tiempo, una estructura desplegable como red tensorial de tipo árbol [10]. Este trabajo es importante para el presente TFG por dos motivos. Primero, identifica de forma explícita que las MPS tradicionales han tenido dificultades para escalar a reconocimiento de imágenes naturales cuando se usan como núcleo principal del modelo. Segundo, conecta las TN con redes polinómicas y multilineales modernas, mostrando que la investigación actual se mueve hacia arquitecturas híbridas, entrenables

con herramientas estándar y mejor adaptadas a datos visuales complejos.

Desde el punto de vista de implementación, también ha habido una evolución clara. Las primeras propuestas de aprendizaje con MPS estaban muy ligadas a algoritmos inspirados en DMRG y a código de investigación específico. Actualmente existen librerías que buscan integrar TN dentro de ecosistemas de aprendizaje profundo. TensorKrowch, por ejemplo, está construido sobre PyTorch y permite definir redes tensoriales entrenables, conectarlas con otros módulos y aprovechar diferenciación automática [11]. Esta disponibilidad de herramientas no elimina las dificultades del problema: siguen siendo decisiones críticas la elección del *embedding*, el orden de las posiciones de entrada de datos, la topología, la dimensión de enlace, la ruta de contracción y la estabilidad del entrenamiento.

En conjunto, el estado del arte muestra que las TN son una línea prometedora para construir modelos compactos, estructurados e interpretables, pero no constituyen todavía una alternativa universal a las arquitecturas dominantes de *machine learning*, como CNNs o Transformers. Su ventaja es más clara cuando se busca controlar el número de parámetros, introducir sesgos estructurales, modelar interacciones multilineales o estudiar arquitecturas inspiradas en física cuántica. Su mayor debilidad práctica aparece al trasladar resultados de datasets sencillos a imágenes reales de mayor resolución, donde la representación espacial y el coste de contracción se vuelven determinantes. Este TFG se sitúa precisamente en ese punto intermedio: parte de modelos consolidados como MPS, estudia alternativas jerárquicas tipo TTN y explora mecanismos híbridos inspirados en trabajos recientes, sin asumir que una red tensorial vaya a superar de forma automática a una red profunda convencional.

1.2 Planteamiento del Problema

El problema que aborda este TFG puede formularse de la siguiente manera: estudiar si una arquitectura basada en Tensor Networks puede utilizarse de forma práctica para resolver una tarea de clasificación de imágenes, manteniendo un número controlado de parámetros y una estructura interpretable, sin depender exclusivamente de arquitecturas profundas convencionales.

Esta formulación general contiene varios retos. El primero es la representación de la imagen. Una imagen digital no es, para una red tensorial, una entrada natural en forma de matriz o mapa de píxeles, sino un objeto que debe convertirse previamente en una secuencia de vectores locales. En datasets simples como el MNIST, esta conversión puede hacerse píxel a píxel. Sin embargo, cuando las imágenes son RGB, tienen mayor resolución y contienen objetos reales con forma, orientación y textura, tratar cada píxel como un sitio tensorial produce secuencias demasiado largas y poco manejables. Por tanto, una parte central del problema consiste en diseñar un *embedding* que reduzca la imagen a una representación compatible con la TN, pero que no destruya por completo la información espacial relevante.

El segundo problema es la elección de la topología tensorial. Una MPS resulta atractiva por su simplicidad y por la madurez de sus algoritmos, pero impone una geometría lineal que no se ajusta de forma natural a una imagen bidimensional. Una PEPS conserva mejor la estructura 2D, pero su contracción exacta es mucho más costosa. Una TTN ofrece una solución intermedia; organiza la información de forma jerárquica, reduce la distancia efectiva entre posiciones de entrada y mantiene una contracción más manejable. Por ello, el proyecto plantea estudiar arquitecturas tensoriales que puedan entrenarse en la práctica y que sean razonables para imágenes reales, aunque no reproduzcan literalmente todos los modelos más complejos de la literatura.

El tercer problema es de implementación. Pasar de una definición matemática de una Tensor Network a un modelo entrenable requiere un nivel de computación elevado. Es necesario construir nodos tensoriales con parámetros, conectar correctamente sus índices, definir una ruta de contracción, integrar la red con PyTorch, calcular gradientes, entrenar por *batches* y registrar métricas. Además, las capas tensoriales deben encajar con el resto del sistema de clasificación: carga de datos, transformaciones, función de pérdida, optimizador, *scheduler*, *checkpoints* y evaluación.

El cuarto problema es experimental. No basta con que el modelo pueda ejecutarse y disminuir la pérdida de entrenamiento; hay que evaluar si aprende patrones visuales que generalicen al conjunto de validación y test. Para ello, el trabajo no puede limitarse a una única métrica final. Es necesario analizar precisión, *macro F1*, curvas de entrenamiento, matrices de confusión, comportamiento por clase y posibles señales de *overfitting* o sobreconfianza. Esta evaluación es especialmente importante porque las TN pueden ser eficientes en parámetros, pero esa eficiencia no garantiza por sí misma un buen rendimiento en clasificación de imágenes.

En la versión final del proyecto, el problema se concreta sobre ChemEq25, un dataset de imágenes de material de laboratorio con 25 clases. Las imágenes originales contienen varios objetos, por lo que el problema se reformula como clasificación de instancias; a partir de las anotaciones se generan recortes individuales, y cada recorte debe clasificarse en una única clase. Así, el sistema completo debe resolver la cadena:

`imagen anotada → crop de objeto → embedding → TN → clase predicha.`

La pregunta principal del trabajo no es si las Tensor Networks sustituyen de manera general a las CNNs o a los Transformers, sino si es posible construir, entrenar y evaluar un clasificador de imágenes basado en TN que funcione de extremo a extremo sobre un problema visual realista, y qué limitaciones aparecen durante ese proceso. A partir de esta pregunta principal se derivan varias cuestiones:

- ¿Qué tipo de representación de imagen permite alimentar una tensor network sin producir secuencias inviables?
- ¿Qué limitaciones aparecen al usar una MPS como punto de partida para imágenes de mayor resolución?

- ¿Puede una estructura jerárquica tipo TTN ofrecer un compromiso razonable entre expresividad y coste computacional?
- ¿Qué aporta combinar contracciones tensoriales con componentes habituales de *deep learning*, como normalización, capas lineales o un *front-end* espacial?
- ¿Qué rendimiento y qué patrones de error se observan al entrenar el modelo sobre un dataset real de clasificación de objetos?

Por tanto, el planteamiento del problema consiste en analizar hasta qué punto las TN son adecuadas para clasificación de imágenes más allá de ejemplos pequeños y controlados.

1.3 Planificación del TFG



Figura 1.1: Planificación por semanas del TFG, cada circulo representa una semana teniendo febrero 3 semanas (porque empiezo el 9 de Febrero), marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.

- **Semanas 1–3: Introducción y fundamentos de Tensor Networks**

Durante las primeras semanas se adquirirán los conocimientos fundamentales y teóricos sobre Tensor Networks.

- Estudio de la notación y representaciones básicas de Tensor Networks mediante material audiovisual.
- Lectura y análisis de artículos científicos relevantes, con especial atención a modelos tensoriales aplicados a clasificación de imágenes.
- Estudio de apuntes y material teórico relacionado con Tensor Networks y su conexión con la física de sistemas cuánticos.

■ **Semanas 4–6: Estudio de modelos MPS para clasificación de imágenes**

En esta etapa se profundizará en el uso de Matrix Product States como modelos de clasificación de imágenes:

- Análisis del funcionamiento de clasificadores basados en MPS y estudio de sus fuentes de error.
- Revisión de implementaciones existentes y comprensión de los procedimientos de entrenamiento y evaluación.
- Búsqueda y selección de conjuntos de datos adecuados en plataformas como Kaggle o Hugging Face.

■ **Semanas 7–8: Formación en herramientas computacionales**

Durante este periodo se adquirirá el conocimiento práctico necesario para el desarrollo del proyecto:

- Aprendizaje de comandos básicos de Bash y trabajo en entornos de servidor.
- Uso de GitHub para el control de versiones y la organización del código.
- Estudio de buenas prácticas de organización de proyectos y documentación.
- Aprendizaje en profundidad de PyTorch para la implementación de modelos de inteligencia artificial.

■ **Semanas 9–13: Implementación y experimentación**

En esta fase se llevará a cabo la implementación y entrenamiento de los modelos seleccionados:

- Implementación de modelos basados en Tensor Networks para clasificación de imágenes.
- Entrenamiento de los modelos utilizando recursos de computación de altas prestaciones (HPC).
- Evaluación del rendimiento de los modelos y comparación con modelos de inteligencia artificial convencionales.

■ **Semanas 13–15: Análisis de resultados y redacción**

En la etapa final se realizará el análisis de los resultados obtenidos y la documentación del trabajo:

- Análisis crítico de los resultados y discusión de las conclusiones.
- Redacción de la memoria del Trabajo Fin de Grado.
- Organización final del repositorio de código y preparación de la entrega.

1.4 Recursos requeridos

A continuación se enumeran los recursos técnicos utilizados para la ejecución del trabajo:

- Recursos computacionales:
 - Servidor HPC Lorca de la Universidad Europea de Madrid: para el entrenamiento de los modelos con recursos de computación de altas prestaciones.
 - Ordenador personal: para el desarrollo, pruebas locales y escritura de la memoria.
- Herramientas de software
 - Python (lenguaje de programación principal).
 - PyTorch (framework de deep learning).
 - TensorKrowch (librería para implementación de Tensor Networks).
 - NumPy, SciPy, Matplotlib, Seaborn (librerías científicas y de visualización).
 - GitHub (control de versiones y gestión del repositorio de código).
 - Bash / Linux (gestión del entorno HPC y automatización de trabajos).
 - Docker / entornos virtuales (gestión de dependencias y reproducibilidad).
- Recursos bibliográficos y de datos:
 - Artículos científicos de referencia (véase sección de Referencias).
 - Conjuntos de datos públicos: MNIST, OlivettiFaces (obtenidos de repositorios como Kaggle o Hugging Face).

Capítulo 2. OBJETIVOS

2.1 Objetivos generales

El objetivo general del trabajo consiste en ampliar el conocimiento actual sobre la aplicación de las Tensor Networks como paradigma alternativo al deep learning convencional en tareas de clasificación de imágenes, evaluando su viabilidad, rendimiento y eficiencia mediante la implementación y experimentación con distintas arquitecturas tensoriales.

Para ello, el proyecto contempla el estudio en profundidad del estado del arte en modelos de inteligencia artificial basados en Tensor Networks, la implementación de al menos dos arquitecturas tensoriales a partir de la literatura científica, su entrenamiento y evaluación utilizando recursos de computación de altas prestaciones (HPC) del servidor Lorca de la Universidad Europea, y la comparación de sus resultados frente a modelos clásicos de deep learning sobre conjuntos de datos de referencia.

De forma complementaria, el proyecto persigue adquirir experiencia práctica en el uso de herramientas computacionales propias del ámbito de la investigación en inteligencia artificial, incluyendo el manejo de entornos HPC, el uso de librerías especializadas como PyTorch y TensorKrowch, y las buenas prácticas de documentación y reproducibilidad del código científico.

2.2 Objetivos específicos

1. Realizar una revisión del estado del arte de los modelos de inteligencia artificial basados en Tensor Networks aplicados a la clasificación de imágenes, analizando las arquitecturas más relevantes y sus principales resultados.
2. Estudiar en detalle distintos modelos tensoriales para clasificación de imágenes, con el fin de comprender su funcionamiento y sus diferencias fundamentales.
3. Implementar al menos dos modelos de inteligencia artificial basados en Tensor Networks para tareas de clasificación de imágenes, a partir de descripciones presentes en la literatura científica.
4. Entrenar y evaluar los modelos implementados utilizando recursos de computación de altas prestaciones (HPC), familiarizándose con el uso de servidores, entornos de ejecución y herramientas básicas de gestión de trabajos.
5. Comparar el rendimiento de los modelos basados en Tensor Networks con modelos de inteligencia artificial convencionales, empleando conjuntos de datos de referencia en clasificación de imágenes.

6. Analizar los resultados obtenidos en términos de precisión, eficiencia computacional y comportamiento del entrenamiento, identificando las principales ventajas y limitaciones de los enfoques tensoriales.
7. Documentar el desarrollo del proyecto, describiendo de forma clara los modelos estudiados, las implementaciones realizadas y los resultados obtenidos.
8. Preparar y organizar un repositorio de código que recoja las implementaciones y experimentos realizados, garantizando su claridad, reproducibilidad y correcta documentación.

2.3 Objetivos que quedan fuera del proyecto

- Estudiar modelos de inteligencia artificial no aplicados a la clasificación de imágenes y que no estén basados en Tensor Networks.
- Evaluar los modelos en conjuntos de datos distintos de los considerados en el proyecto, o realizar comparaciones extensivas en múltiples benchmarks adicionales.
- Ejecutar los modelos en hardware cuántico real, más allá de enfoques cuántico-inspirados ejecutados en sistemas clásicos.
- Desarrollar aplicaciones orientadas a entornos industriales o sistemas en producción a partir de los modelos estudiados.

2.4 Beneficios del proyecto

Este proyecto contribuye a ampliar el conocimiento empírico sobre las Tensor Networks como herramienta de aprendizaje automático. Si bien el fundamento teórico de estas arquitecturas está bien establecido en el ámbito de la física cuántica y matemática, su comportamiento práctico en tareas de clasificación de imágenes sigue siendo un área activa de investigación. El trabajo aportará evidencia experimental concreta sobre estas cuestiones.

Desde el punto de vista computacional, el proyecto favorece el desarrollo de modelos de IA más eficientes en el uso de parámetros. Los modelos basados en Tensor Networks permiten, en principio, representar funciones complejas con un número significativamente menor de parámetros que las redes neuronales profundas convencionales. Esto tiene implicaciones directas sobre el coste energético y computacional del entrenamiento, un aspecto de creciente relevancia en la investigación en inteligencia artificial.

La realización de este proyecto supone una experiencia muy íntegra en investigación aplicada: comprende la revisión crítica de literatura científica, el diseño e implementación de experimentos reproducibles, el uso de infraestructuras HPC y el aprendizaje de herramientas y flujos de trabajo propios del ámbito profesional de la investigación en IA. Estos conocimientos y competencias son directamente transferibles a entornos académicos y laborales en el campo de la física y la computación científica.

Capítulo 3. MARCO TEÓRICO

3.1 Deep learning y clasificación de imágenes

Antes de explicar el concepto de *deep learning* es importante entender que es una red neuronal. Una red neuronal es una estructura computacional formada por un conjunto de unidades de procesamiento simples, denominados neuronas. Éstas están interconectadas mediante pesos sinápticos ajustables, llamados en inglés *weights* y *biases*. Cada neurona recibe una serie de entradas, a las cuales llamaremos *inputs*; se calcula una suma ponderada de las mismas y se les aplican una función de activación para producir una salida, la cual llamaremos *output*. El comportamiento de la red, queda determinado por los valores de estos pesos, que son modificados durante el entrenamiento con el objetivo de minimizar una función de error entre los *outputs* y las respuestas esperadas.

Formalizando esta descripción, dada una entrada $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, la operación de una única neurona se expresa como:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \quad (3.1)$$

donde $\mathbf{w} = (w_1, w_2, \dots, w_n)$ es el vector de pesos sinápticos, b es el sesgo (*bias*), y $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación. El argumento de σ es una transformación afín de la entrada: un producto escalar seguido de una traslación. La no linealidad introducida por σ es esencial, pues sin ella la composición de múltiples capas lineales se reduciría a una única transformación lineal, independientemente de la profundidad de la red [12].

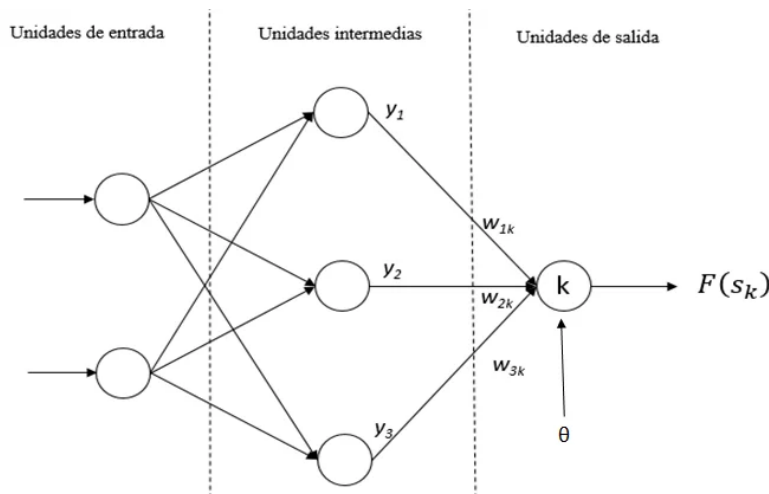


Figura 3.1: Esquema de una red neuronal simple. [1]

Las funciones de activación más utilizadas en la práctica incluyen la sigmoide $\sigma(z) = 1/(1+e^{-z})$, la tangente hiperbólica $\sigma(z) = \tanh(z)$, y la unidad lineal rectificada (ReLU) $\sigma(z) = \max(0, z)$. La elección de la función de activación tiene consecuencias importantes sobre la dinámica del entrenamiento: funciones como la sigmoide pueden producir gradientes que se desvanecen en redes profundas, mientras que la ReLU mitiga parcialmente este problema [12].

Una capa completa de la red consiste en m neuronas que operan en paralelo sobre la misma entrada, produciendo un vector de salida $\mathbf{y} \in \mathbb{R}^m$:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad (3.2)$$

donde $\mathbf{W} \in \mathbb{R}^{m \times n}$ es la matriz de pesos de la capa y $\mathbf{b} \in \mathbb{R}^m$ el vector de sesgos.

Llegamos a la definición de *deep learning*, que no es más que una red neuronal con muchas capas intermedias. La expresión matemática que define una *deep learning* es una extensión de la Ecuación 3.2 pero para L capas:

$$f(\mathbf{x}) = \sigma_L(\mathbf{W}_L \cdots \sigma_2(\mathbf{W}_2 \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots + \mathbf{b}_L) \quad (3.3)$$

Los parámetros entrenables del modelo son el conjunto $\{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \dots, \mathbf{W}_L, \mathbf{b}_L\}$, cuyo número total crece con la anchura y la profundidad de la red.

El entrenamiento de una red neuronal se formula como un problema de optimización. Dado un conjunto de entrenamiento $\{(\mathbf{x}_n, y_n)\}_{n=1}^{N_T}$, se define una *función de coste* (o función de pérdida) C que mide la discrepancia entre las predicciones del modelo $f(\mathbf{x}_n)$ y las etiquetas verdaderas y_n . El objetivo es encontrar los parámetros $\boldsymbol{\theta} = \{\mathbf{W}_l, \mathbf{b}_l\}$ que minimizan C :

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \frac{1}{N_T} \sum_{n=1}^{N_T} \mathcal{L}(f(\mathbf{x}_n; \boldsymbol{\theta}), y_n) \quad (3.4)$$

donde $\mathcal{L}$ es una función de pérdida por muestra (por ejemplo, el error cuadrático medio o la entropía cruzada). Este es un problema de minimización en un espacio de alta dimensión, que se resuelve mediante variantes del *descenso de gradiente* [12].

El algoritmo fundamental para calcular los gradientes necesarios es la *backpropagation*, que aplica sistemáticamente la regla de la cadena del cálculo diferencial para propagar el error desde la capa de salida hacia las capas inferiores. Para un parámetro θ cualquiera de la red, el gradiente se calcula como:

$$\frac{\partial C}{\partial \theta} = \frac{\partial C}{\partial \mathbf{y}_L} \cdot \frac{\partial \mathbf{y}_L}{\partial \mathbf{y}_{L-1}} \cdots \frac{\partial \mathbf{y}_{l+1}}{\partial \mathbf{y}_l} \cdot \frac{\partial \mathbf{y}_l}{\partial \theta} \quad (3.5)$$

donde $\mathbf{y}_l$ es la salida de la capa l -ésima. La actualización de los parámetros sigue entonces la regla $\theta \leftarrow \theta - \eta \partial C / \partial \theta$, donde $\eta > 0$ es la tasa de aprendizaje.

La diferencia fundamental entre las redes clásicas y las redes *deep* radica en su capacidad de representación. Una red con una única capa oculta puede, en teoría, aproximar cualquier función continua con la suficiente anchura, pero resulta ineficiente para capturar estructuras jerárquicas, como las que están presentes en las imágenes o en el lenguaje natural [5]. Las *deep learning*, en cambio, aprenden representaciones en múltiples niveles de abstracción: las capas más bajas detectan características primitivas como bordes o frecuencias, mientras que las superiores combinan estas primitivas para reconocer objetos complejos o conceptos semánticos [4].

En clasificación de imágenes, las redes convolucionales han sido especialmente relevantes porque incorporan localidad espacial y pesos compartidos. Estas dos propiedades son adecuadas para datos visuales: los píxeles vecinos tienden a estar relacionados, y los mismos patrones pueden aparecer en distintas regiones de la imagen. La Figura 3.2 muestra algunas topologías básicas de redes neuronales.

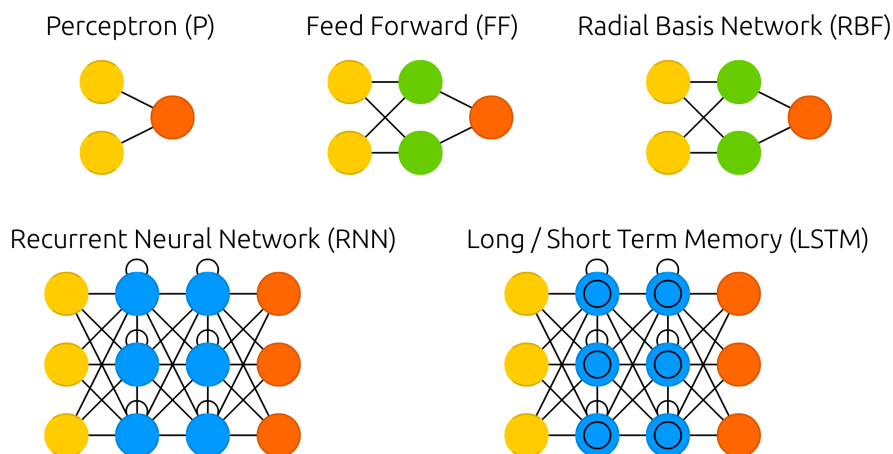


Figura 3.2: Esquema de topologías básicas de redes neuronales, desde el perceptrón hasta arquitecturas con capas ocultas y conexiones recurrentes [2].

El éxito de estas arquitecturas no elimina sus limitaciones. Muchos modelos modernos alcanzan buenos resultados con un número elevado de parámetros y con un coste computacional considerable. Este TFG se sitúa en ese punto; estudia si las TN pueden proporcionar modelos de clasificación estructurados, capaces de representar funciones complejas mediante factorizaciones tensoriales controladas.

3.2 Tensores

Antes de profundizar en el mundo de las TN es conveniente sentar primero las bases. Lo primero es entender que un tensor. Un tensor lo podemos ver definido de diferentes formas dependiendo del campo en el que estemos trabajando.

Desde un puesto de vista computacional, los tensores son *arrays* multidimensionales que

generalizan de forma natural los conceptos de escalar, vector y matriz a dimensiones arbitrariamente superiores. Desde un punto de vista físico-matemático, los tensor son objetos matemáticos de orden superior, que representan relaciones multidimensionales entre distintas variables o grados de libertad. Dicho de otra forma, un tensor de orden N es un objeto cuyos elementos tienen N índices y están asociados a N vectores base.

$$R^{I_1 \times I_2 \times \dots \times I_N}$$

donde N es el número de modos y las longitudes $I_1, \dots, I_N$ representan las dimensiones de cada modo. [4]

Bajo esta definición, los escalares son tensores de orden cero, los vectores son tensores de orden 1, y las matrices son tensores de orden 2; todo caso superior constituye propiamente lo que en la literatura se denomina, tensor de alto orden. En la notación convencional, se distinguen los diferentes objetos según su orden: las letras minúsculas en cursiva (a) denotan escalares, las letras minúsculas en negrita ($\mathbf{a}$) denotan vectores, las letras mayúsculas en negrita ($\mathbf{A}$) denotan matrices, y las letras mayúsculas en tipografía cursiva en negrita ($\mathcal{A}$) denotan tensores de orden superior. [4]

La intuición física detrás de los tensores resulta natural cuando se considera, que muchos fenómenos del mundo real son inherentemente multidimensionales. Una imagen en color, por ejemplo, es un tensor de orden tres: altura, anchura y canales cromáticos. Los datos de resonancia magnética funcional (fMRI) constituyen tensores de orden cuatro. En mecánica cuántica, las funciones de onda variacionales de sistemas de muchos cuerpos son, igualmente, tensores de alto orden [4].

El concepto de tensor tiene raíces en el siglo XIX, ligado al desarrollo del cálculo diferencial absoluto por Ricci-Curbastro y Levi-Civita, aunque fue la formulación de la relatividad general por Einstein en 1915 la que consagró los tensores como herramienta fundamental de la física matemática. La descripción matemática de un Tensor es la siguiente:

$$\mathcal{T} = \sum_{i_1, i_2, \dots, i_n} T_{i_1 i_2 \dots i_n} \hat{b}_{i_1} \hat{b}_{i_2} \dots \hat{b}_{i_n} \quad (3.6)$$

La operación central en el álgebra tensorial es la contracción tensorial. Dados dos tensores $\mathcal{T}$ y $\mathcal{R}$, de orden n y m respectivamente, con elementos $T_{i_1 i_2 \dots i_n}$ y $R_{j_1 j_2 \dots j_m}$ y en la misma base ortonormal $\hat{b}_i$, diremos que el tensor $\mathcal{C}$ es la contracción de ambos tensores por sus último q índices cuando:

$$\mathcal{C} = \sum_{i_1, i_2, \dots, i_{n-q}, j_1, j_2, \dots, j_{m-q}} C_{i_1 i_2 \dots i_{n-q}, j_1 j_2 \dots j_{m-q}} \hat{b}_{i_1} \hat{b}_{i_2} \dots \hat{b}_{i_{n-q}} \hat{b}_{j_1} \hat{b}_{j_2} \dots \hat{b}_{j_{m-q}} \quad (3.7)$$

siendo sus elementos

$$C_{i_1 i_2 \dots i_{n-q}, j_1 j_2 \dots j_{m-q}} = \sum_{k_1, k_2, \dots, k_q} T_{i_1 i_2 \dots i_{n-q}, k_1 k_2 \dots k_q} R_{j_1 j_2 \dots j_{m-q}, k_1 k_2 \dots k_q} \quad (3.8)$$

De esta definición podemos sacar que la contracción tensorial es la operación mediante la cual dos tensores se contraen en un único tensor a lo largo de sus pares de índices asociados. Existen formas diferentes de contracciones, algunas implican más de dos tensores o un índice repetido en más de dos tensores o incluso repetidos en el mismo tensor. El caso más familiar de contracción tensorial es la multiplicación matricial:

$$\mathbf{AB} = \sum_k A_{ik} B_{kj} \hat{b}_i \hat{b}_j$$

o el producto escalar entre dos vectores:

$$\vec{v} \cdot \vec{w} = \sum_i v_i w_i$$

Entre las operaciones tensoriales fundamentales cabe destacar la traza de una matriz:

$$Tr(A) = \sum_i A_{ii} \quad (3.9)$$

Esto sería la contracción de una matriz consigo misma a través de sus índices repetidos. Debido a esta operación podemos ver que, un tensor también sirve como mapeo entre dos espacios vectoriales. Esto es un tensor, que recibe otro tensor y devuelve un tercer tensor. Por cada índice contraído estamos mezclando componentes del tensor de entrada, reduciendo en 1 el orden del tensor de salida. Por otro lado, los índices que no han sido contraídos aumentarán en 1 el orden del tensor de salida.

3.3 Notación diagramática de Penrose y definición de Tensor Network

La representación algebraica de las contracciones tensoriales, aunque precisa, se vuelve rápidamente inmanejable a medida que aumenta el número de tensores y la complejidad de sus conexiones. Fue Roger Penrose [\[13\]](#) quien, a principios de la década de 1970, introdujo la notación diagramática para tensores, proporcionando una alternativa visual que transformó la manera en que se razona sobre estas estructuras. Los diagramas de redes tensoriales son hoy ampliamente utilizados para describir algoritmos cuánticos y algoritmos de *machine learning* [\[4\]](#).

La idea central de la notación de Penrose es representar cada tensor como un nodo del que emanan tantas aristas como índices tenga el tensor. Concretamente: un nodo con una sola arista representa un vector $\mathbf{a} \in \mathbb{R}^I$; un nodo con dos aristas representa una matriz

$\mathbf{A} \in \mathbb{R}^{I \times J}$; y un nodo con tres aristas representa un tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$. Para indicar que un par de índices tensoriales está siendo contraído, ambas aristas se unen mediante una línea. Por ejemplo, la contracción de un tensor de orden uno con uno de orden dos es la multiplicación matriz-vector habitual. Las aristas que no se conectan a ningún otro tensor se denominan aristas libres, y el número de estas aristas en el diagrama resultante es precisamente el orden del tensor final. Dos nodos que no están unidos mediante aristas, representan el producto tensorial entre ellos.

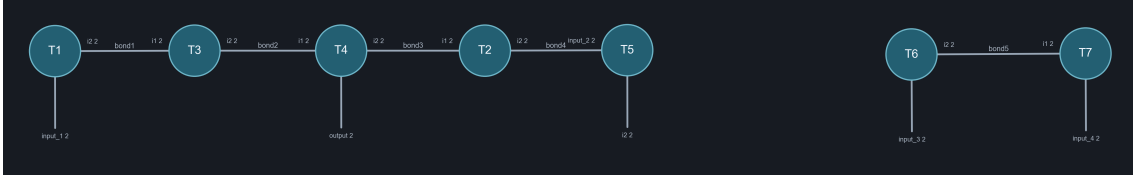


Figura 3.3: Representación de una TN inventada. En esta representación se pueden identificar muchas de las operaciones descritas en esta sección. Elaboración propia.

La notación gráfica tensorial ofrece numerosas ventajas sobre la notación tradicional: en forma gráfica, los índices no necesitan nombres ni etiquetas; operaciones como el producto escalar, la traza tensorial y la contracción pueden expresarse sin notación adicional; y el orden del tensor resultante puede leerse simplemente contando el número de aristas libres. En particular, si en el diagrama no quedan aristas libres tras todas las contracciones, el resultado es necesariamente un escalar. Esta propiedad convierte los diagramas en una herramienta de verificación inmediata de la consistencia dimensional de operaciones complejas [6]. Sobre esta base visual puede introducirse ya, de manera natural, el concepto de *Tensor Network (TN)*.

Una TN es, en esencia, una representación gráfica de una ecuación multilineal entre diferentes tensores. Esta representación se apoya en la notación diagramática de Penrose, que con el tiempo se ha ido adaptando a las necesidades de disciplinas como la física, las matemáticas o la ciencia computacional [14]. Puede verse un ejemplo en la Figura 3.4, donde aparece la TN que representa el 2-tensor T_{ip} cuyos elementos se obtienen mediante la operación de contracción

$$T_{ip} = \sum_{j,k,l,m,n,o} A_{ijkl} B_{jmn} C_{kmn} D_{lo} E_{nop} \quad (3.10)$$

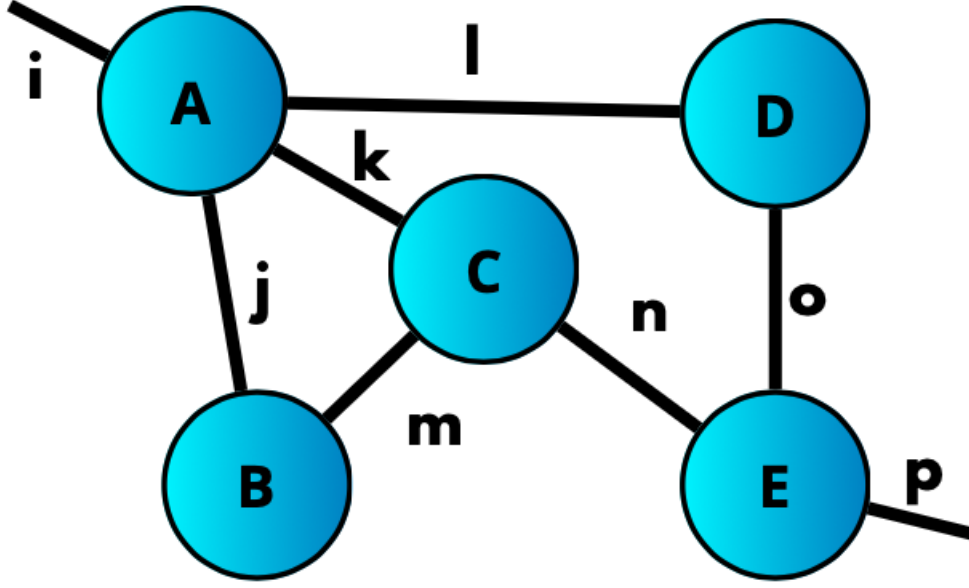


Figura 3.4: Ejemplo de representación diagramática para un tensor. Fuente: [3]

La motivación fundamental para el uso de las Tensor Networks es de naturaleza computacional. Un tensor de orden N con dimensión d en cada nodo tiene d^N componentes, un número que crece exponencialmente con el orden del tensor, haciendo que su almacenamiento y manipulación directos sean computacionalmente prohibitivos; este fenómeno se conoce como la *maldición de la dimensionalidad*. Las TN abordan este problema representando el tensor original no de forma explícita, sino como el resultado de contraer un conjunto de tensores de menor orden, cada uno con un número de parámetros manejable, de modo que la complejidad resultante escala de forma polinomial en lugar de exponencial [4, 6]. Esta idea general se entiende mejor cuando se examina el contexto físico en el que surgió.

3.4 Tensor Networks y reducción de dimensionalidad

Una Tensor Network puede entenderse como una factorización de un tensor de alto orden en una colección de tensores de menor orden conectados entre sí. Su utilidad aparece cuando el tensor completo tendría un número de componentes inmanejable. Si un tensor tiene N modos y cada modo tiene dimensión d , representarlo de forma explícita requiere d^N componentes. Este crecimiento exponencial se conoce como *maldición de la dimensionalidad*.

En física cuántica, el mismo fenómeno aparece al describir sistemas de muchas partículas. Un estado cuántico general de N subsistemas con dimensión local d tiene coeficientes:

$$|\psi\rangle = \sum_{s_1, \dots, s_N} C_{s_1 \dots s_N} |s_1\rangle \otimes \dots \otimes |s_N\rangle. \quad (3.11)$$

El tensor $C_{s_1 \dots s_N}$ tiene d^N componentes. Sin embargo, muchos estados físicamente relevantes poseen una estructura interna que permite representarlos de forma compacta. La idea clave es que no todos los estados ocupan todo el espacio posible con la misma complejidad; sus correlaciones o su entrelazamiento pueden estar limitados [15].

Esta observación se traslada al *machine learning* como una analogía útil. Un modelo de clasificación no necesita representar una función arbitraria sobre todos los píxeles posibles, sino una función que aproveche regularidades de los datos. Las Tensor Networks imponen una estructura de bajo rango o de conectividad concreta sobre el tensor de pesos, reduciendo el número de parámetros si las dimensiones internas, llamadas dimensiones de enlace, permanecen controladas [5, 4].

3.5 Descomposiciones tensoriales fundamentales

Para comprender cómo se construyen las representaciones mediante TN, conviene introducir primero algunas descomposiciones tensoriales básicas. La herramienta algebraica fundamental es la descomposición en valores singulares (SVD). Dada una matriz $\mathbf{A} \in \mathbb{R}^{M \times N}$, la SVD la factoriza como

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top, \quad (3.12)$$

donde $\mathbf{U}$ y $\mathbf{V}$ son matrices ortogonales, y $\mathbf{\Sigma}$ contiene los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. La importancia de la SVD en TN se debe a que permite obtener la mejor aproximación de rango bajo en norma de Frobenius. Al conservar solo los R valores singulares dominantes se obtiene la matriz de rango R que minimiza el error de reconstrucción, resultado conocido como teorema de Eckart–Young [4].

Una primera generalización del rango matricial a tensores de orden superior es la descomposición CP. Dado un tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$, CP lo aproxima como suma de R tensores de rango 1:

$$\mathcal{X} \approx \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}, \quad (3.13)$$

donde $\circ$ denota el producto exterior y R es el rango CP. Esta representación es muy compacta, con $R \sum_{n=1}^N I_n$ parámetros, pero presenta dificultades prácticas; determinar el rango CP es un problema NP-difícil y los métodos de ajuste habituales no garantizan alcanzar el óptimo global [16, 4]. En notación diagramática, CP se representa mediante factores vectoriales conectados a un nodo central diagonal.

La descomposición de Tucker es más flexible. En lugar de una única suma de componentes de rango 1, introduce un tensor núcleo $\mathcal{G}$ que modela las interacciones entre los factores:

$$\mathcal{X}_{i_1 i_2 \dots i_N} \approx \sum_{r_1=1}^{R_1} \dots \sum_{r_N=1}^{R_N} \mathcal{G}_{r_1 r_2 \dots r_N} A_{i_1 r_1}^{(1)} A_{i_2 r_2}^{(2)} \dots A_{i_N r_N}^{(N)}. \quad (3.14)$$

Los valores $R_1, \dots, R_N$ son los rangos de Tucker y pueden ser distintos para cada modo. Si las matrices de factores son ortogonales, se obtiene la Higher-Order Singular Value Decomposition (HOSVD), una extensión natural de la SVD al caso tensorial [4]. CP puede verse como un caso particular de Tucker cuando el núcleo es superdiagonal y todos los rangos coinciden. Sin embargo, el núcleo de Tucker contiene $\prod_{n=1}^N R_n$ parámetros, por lo que el coste vuelve a crecer exponencialmente con el orden del tensor.

Esta limitación motiva formatos más escalables como Tensor Train (TT), conocido en física como Matrix Product State (MPS) [17, 15]. TT representa un tensor de orden N como una cadena de tensores locales $\mathcal{G}^{(n)} \in \mathbb{R}^{R_{n-1} \times I_n \times R_n}$ conectados mediante índices virtuales. Su número de parámetros es

$$\sum_{n=1}^N R_{n-1} I_n R_n,$$

que crece linealmente con N si los rangos se mantienen acotados. Además, la descomposición TT puede construirse mediante SVD sucesivas, lo que proporciona estabilidad numérica y permite controlar el error mediante el truncamiento de valores singulares [4, 17]. Desde la perspectiva cuántica, los rangos TT coinciden con los rangos de Schmidt de las biparticiones correspondientes, conectando esta construcción con la noción de entrelazamiento discutida anteriormente [15].

La Tabla 3.1 resume las diferencias principales entre los formatos anteriores.

Formato	Parámetros	Observaciones
CP	$R \sum_n I_n$	Muy compacta, pero el rango CP es difícil de determinar.
Tucker	$\prod_n R_n + \sum_n I_n R_n$	Flexible, pero el núcleo crece exponencialmente con el orden.
TT/MPS	$\sum_n R_{n-1} I_n R_n$	Escalado lineal para rangos fijos; construible mediante SVD sucesivas.

Tabla 3.1: Comparación entre las descomposiciones CP, Tucker y TT/MPS.

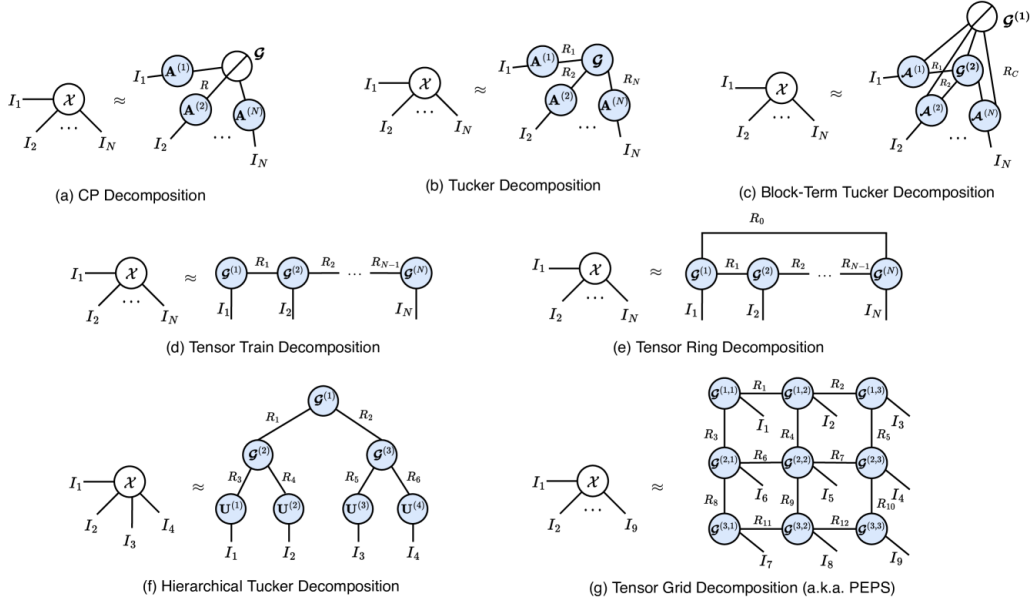


Figura 3.5: Diagramas TN de algunas descomposiciones TN populares. (a) El formato CP descompone un tensor X en una suma de varios tensores de rango 1 $a_{:,r}^{(1)} \circ a_{:,r}^{(2)} \circ \dots \circ a_{:,r}^{(N)}$. (b) La descomposición de Tucker descompone un tensor X en un tensor núcleo G multiplicado por una matriz $A^{(n)}$ a lo largo del modo n -ésimo. (c) La descomposición en términos de bloque descompone un tensor X en una suma de varias descomposiciones de Tucker, con rangos de Tucker bajos. (d) La descomposición TT descompone un tensor X en una multiplicación lineal de un conjunto de tensores núcleo de tercer orden $G^{(2)} \dots G^{(N-1)}$ y dos matrices $G^{(1)}$ y $G^{(N)}$. (e) La descomposición TR descompone un tensor X en un conjunto de tensores núcleo de tercer orden y los contrae en una estructura de anillo. (f) La descomposición HT representa un tensor X como un diagrama con estructura de árbol. (g) La descomposición en malla tensorial, también conocida como PEPS, representa un tensor de alta dimensión como una malla bidimensional de tensores interconectados de menor rango, donde cada nodo se conecta con sus vecinos para capturar eficientemente correlaciones espaciales en sistemas con interacciones locales. [4]

3.6 Topologías principales de Tensor Networks

La topología de una Tensor Network determina qué correlaciones puede representar de forma eficiente y con qué coste computacional. En esta sección se detallan las topologías de TN más utilizadas y más relevantes en la literatura.

3.6.1 Matrix Product State / Tensor Train

El *Matrix Product State* (MPS), o Tensor Train (TT), es la topología de TN más estudiada y la que mejor se entiende. Su estructura consiste en una cadena lineal de N tensores de orden 3 conectados secuencialmente a través de índices virtuales de dimensión D (dimension de enlace).

El MPS es la representación más natural de los estados fundamentales de los sistemas cuánticos unidimensionales con interacciones locales, lo que lo hace bastante famoso en el campo de la física.

Las correlaciones que un MPS puede representar decaen exponencialmente con la distancia entre sitios: la función de correlación entre los sitios i y j satisface $\langle O_i O_j \rangle - \langle O_i \rangle \langle O_j \rangle \sim e^{-|i-j|/\xi}$, donde ξ es una longitud de correlación que depende de la dimensión de enlace D [6]. Esto significa que el MPS es especialmente eficiente para capturar correlaciones locales o de corto alcance, pero puede requerir dimensiones de enlace elevadas para representar correlaciones de largo alcance.

El MPS es muy relevante porque combina una interpretación física sólida, algoritmos de contracción manejables e implementaciones maduras. En aprendizaje supervisado, Stoudenmire y Schwab lo utilizaron para parametrizar clasificadores de imágenes sobre MNIST, alcanzando error de test inferior al 1 % en ese conjunto [6]. Su limitación principal es que impone una geometría unidimensional, y una imagen 2D debe linealizarse antes de entrar en la red.

3.6.2 Tree Tensor Network

Una *Tree Tensor Network* (TTN) adopta una estructura jerárquica. Esta estructura, en lugar de propagar la información a lo largo de una cadena, asocia los tensores de la capa más baja (lo que en la literatura se denomina las *hojas* o *leaves* en inglés) a los índices del tensor original. Posteriormente se van contrayendo por pares hacia tensores de capas superiores, reduciendo progresivamente el número de índices hasta llegar a un único tensor raíz, que produce la salida. La distancia entre dos hojas del árbol suele crecer como $\mathcal{O}(\log N)$, lo que puede facilitar la combinación de información lejana respecto a una MPS lineal [4, 9].

Esta propiedad, hace a la TTN especialmente atractiva para este trabajo, porque esta estructura es mucho más eficiente que el MPS para capturar correlaciones de largo alcance. En física, el TTN se utiliza para representar estados con una estructura de entrelazamiento jerárquica, como la que aparece en ciertos sistemas con simetría de escala. En *machine learning*, la estructura de árbol resulta natural para datos con organización jerárquica intrínseca [4].

Sin embargo, el TTN tiene una limitación. Al ser una estructura de árbol sin ciclos, la información entre dos ramas distintas solo puede propagarse a través del nodo ancestro común. Esto limita la capacidad de la red para capturar correlaciones entre nodos que se encuentran en ramas separadas del árbol. Esta restricción motivó el desarrollo de arquitecturas más sofisticadas como la MERA.

3.6.3 MERA

La *Multi-scale Entanglement Renormalization Ansatz* (MERA) fue introducida por Vidal en 2007 como una Tensor Network capaz de representar eficientemente los estados fundamentales de sistemas cuánticos críticos, es decir, sistemas en los que las correlaciones decaen como una ley de potencias en lugar de exponencialmente [18].

La innovación clave de la MERA respecto al TTN es la inclusión de tensores adicionales denominados *disentanglers* (desentrelazadores). En cada capa de la estructura jerárquica, antes de aplicar los tensores de renormalización, que reducen el número de grados de libertad, se aplican *disentanglers* que actúan sobre pares de sitios vecinos para eliminar el entrelazamiento de corto alcance. Los tensores de renormalización se encargan de capturar las correlaciones a escalas mayores. La alternancia de capas de *disentanglers* y capas de renormalización permite a la MERA representar correlaciones a todas las escalas de longitud simultáneamente [4].

La topología MERA destaca por su capacidad para describir de forma eficiente sistemas con correlaciones a múltiples escalas, como ocurre típicamente en sistemas cuánticos complejos. En estos casos, una representación basada en MPS puede requerir una dimensión de enlace que crece de forma polinómica con el tamaño del sistema, mientras que la estructura jerárquica de la MERA permite capturar estas correlaciones de manera más natural [18]. Esta organización por capas está estrechamente relacionada con el concepto de grupo de renormalización, en el que cada nivel de la red puede interpretarse como una descripción efectiva del sistema a una escala de longitud diferente.

En *machine learning*, la topología MERA resulta prometedora para el procesamiento de datos con dependencias a múltiples escalas, como puede ser el caso de las imágenes [4]. Sin embargo, su mayor complejidad estructural (más parámetros y contracciones más costosas que el MPS o el TTN) hace que su uso práctico sea más limitado hasta la fecha.

3.6.4 PEPS

Los *Projected Entangled Pair States* (PEPS) son la generalización más natural del MPS a dos dimensiones. Mientras que en la topología MPS cada tensor se conecta a lo sumo con dos vecinos, en PEPS cada tensor se conecta con cuatro vecinos, formando una red bidimensional que refleja directamente la geometría de una rejilla cuadrada [19].

Para un sistema bidimensional de $M \times N$ sitios, cada tensor local en PEPS tiene un índice físico de dimensión d y cuatro índices virtuales (arriba, abajo, izquierda, derecha) de dimensión D . Esto permite capturar de forma natural las correlaciones espaciales bidimensionales que caracterizan, por ejemplo, los estados fundamentales de sistemas cuánticos definidos sobre rejillas planas [4].

Sin embargo, esta ventaja geométrica tiene un coste. La contracción exacta de una red PEPS es un problema computacionalmente intratable; a diferencia del MPS, donde la contracción exacta tiene un coste polinomial. En la práctica, esto obliga a utilizar métodos

de contracción aproximada, lo que introduce errores adicionales y limita el tamaño de las dimensiones de enlace que pueden utilizarse [4]. Existen aplicaciones de PEPS a clasificación supervisada en conjuntos como MNIST y Fashion-MNIST [7].

Topología	Parámetros típicos	Ventaja principal	Limitación práctica
MPS / TT	$\mathcal{O}(NdD^2)$	Contracción eficiente y literatura consolidada.	Geometría 1D para datos que pueden ser 2D.
TTN	$\mathcal{O}(ND^3)$	Agregación jerárquica y distancia logarítmica.	Interacciones limitadas entre ramas separadas.
PEPS	$\mathcal{O}(NdD^4)$	Geometría 2D natural.	Contracción exacta intratable en general.
MERA	$\mathcal{O}(ND^4)$	Correlaciones multiescala.	Mayor complejidad estructural y computacional.

Tabla 3.2: Comparativa sintética de topologías relevantes de Tensor Networks.

Capítulo 4. METODOLOGÍA

En los capítulos anteriores se han introducido los conceptos necesarios para entender los conceptos matemáticos que subyacen el *machine learning*, el *deep learning* y las TN. Sin embargo, para explicar el desarrollo completo de este TFG hace falta cambiar el enfoque. En esta sección se explicará cómo se construye un experimento completo de clasificación de imágenes, qué problemas aparecieron al llevar esa teoría al código y por qué la arquitectura final termina teniendo la forma que tiene.

Este cambio de punto de vista es necesario porque una parte considerable del trabajo no consistió únicamente en proponer una arquitectura, sino en hacer que todo el sistema funcionara de extremo a extremo. En un problema de clasificación de imágenes intervienen varias piezas que tienen que encajar entre sí; la preparación del dataset, el preprocesamiento de las imágenes, la carga de datos por *batches*, la construcción del modelo, el entrenamiento, la evaluación y el registro de resultados. En este proyecto todas esas piezas se implementaron en Python, usando PyTorch como librería principal para construir modelos diferenciables y entrenarlos mediante *backpropagation*.

Por tanto, este capítulo tiene dos objetivos. El primero es explicar la estructura computacional del experimento; cómo se cargan los datos, cómo se transforman las imágenes, cómo se define el modelo y cómo se entrena. El segundo es reconstruir la evolución del proyecto; por qué se empezó con una idea más simple, qué limitaciones se fueron encontrando y cómo esas limitaciones llevaron a la versión final basada en un front-end AIM y una TTN híbrida propia. Esta segunda parte es especialmente importante, porque la arquitectura final no apareció de golpe, sino como resultado de una serie de decisiones tomadas a partir de pruebas, errores y diagnósticos experimentales.

4.1 Pipeline computacional de clasificación en PyTorch

Antes de entrar en los detalles concretos del modelo tensorial, conviene describir la estructura general del pipeline de clasificación. En PyTorch, un experimento de clasificación de imágenes suele organizarse alrededor de una cadena bastante estable; un dataset define cómo se accede a cada muestra, un conjunto de transformaciones prepara la imagen, un *DataLoader* agrupa las muestras en *batches*, un modelo genera predicciones, una función de pérdida compara esas predicciones con la etiqueta correcta y un optimizador actualiza los parámetros. Esta estructura práctica complementa la visión general de entrenamiento, regularización y optimización de modelos profundos descrita en [20].

```
dataset ->transforms ->DataLoader ->modelo ->loss ->backward ->optimizer
```

En este proyecto, el dataset no se entiende simplemente como una carpeta con imágenes. Cada muestra debe proporcionar una imagen y una etiqueta asociada, de forma que el modelo pueda aprender a relacionar una entrada visual con una clase concreta. Además, en el caso del dataset principal del proyecto, algunas de las imágenes originales contienen varios objetos de laboratorio en una misma escena. Por este motivo, antes de entrenar el clasificador fue necesario convertir las anotaciones del dataset en muestras individuales mediante recortes o *crops* de cada objeto.

El preprocesamiento transforma cada imagen a un formato común; tamaño fijo, conversión a tensor, normalización y, cuando procede, aumento de datos. Después, el *DataLoader* permite recorrer el dataset por *batches*, mezclar los ejemplos de entrenamiento y separar correctamente los conjuntos de entrenamiento, validación y test.

El modelo recibe un *batch* de imágenes y devuelve una puntuación para cada clase. A esas puntuaciones se les llama *logits*. En clasificación multiclase no se aplica una *softmax* al final del modelo, porque la función `CrossEntropyLoss` de PyTorch ya combina internamente una normalización logarítmica con la comparación frente a la etiqueta correcta. Una vez calculada la pérdida, PyTorch obtiene los gradientes con `loss.backward()` y el optimizador actualiza los parámetros con `optimizer.step()`.

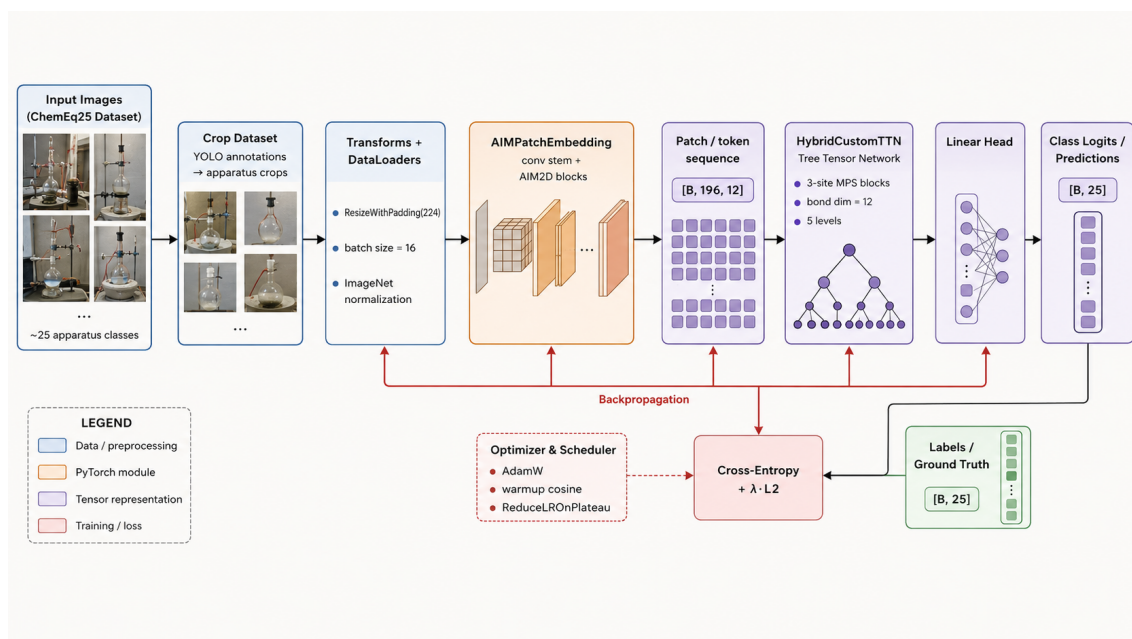


Figura 4.1: Esquema general del pipeline desarrollado en el TFG para la clasificación de imágenes del dataset ChemEq25. A partir de las imágenes originales y sus anotaciones se generan recortes individuales de objetos, que posteriormente se transforman, normalizan y agrupan en *batches*. Cada imagen procesada se introduce en un `AIMPatchEmbedding`, que conserva parte de la estructura espacial mediante un *stem* convolucional y bloques AIM2D antes de convertirla en una secuencia de *patches* de dimensión $[B, 196, 12]$. Esta secuencia alimenta la arquitectura `HybridCustomTTN`, formada por bloques MPS de tres sitios organizados jerárquicamente con dimensión de enlace `bond_dim = 12`. La salida de la TN pasa por una cabeza lineal que produce los *logits* de clasificación para las 25 clases. La parte inferior del diagrama representa el proceso de entrenamiento; comparación con las etiquetas reales mediante entropía cruzada regularizada con término L_2 , actualización con AdamW y planificación del aprendizaje mediante *warmup* coseno y `ReduceLROnPlateau`. Las flechas rojas indican la *backpropagation* de los gradientes a través de los módulos entrenables.

4.2 Organización del proyecto

Una vez fijado el pipeline general, el código se organizó separando las distintas responsabilidades del experimento. Esta organización fue necesaria para poder cambiar una parte del sistema sin tener que reescribir todas las demás. Por ejemplo, se podría cambiar el *embedding* sin modificar el bucle de entrenamiento, o sustituir una MPS por una TTN manteniendo la misma interfaz de entrada y salida.

```
src/tn_dl/
cli/           puntos de entrada para entrenar, evaluar o auditar
config/       lectura y validación de parámetros desde YAML
data/         datasets, splits, crops, transforms y DataLoaders
```

<code>embeddings/</code>	transformaciones de imagen a secuencia tensorial
<code>layers/</code>	capas MPS, TTN, HybridCustomTTN y AIM
<code>models/</code>	composición embedding + clasificador tensorial
<code>training/</code>	bucle de entrenamiento, loss, optimizador y scheduler
<code>evaluation/</code>	diagnósticos, matrices de confusión y auditorías
<code>utils/</code>	utilidades generales del proyecto
<code>configs/</code>	ficheros YAML de experimentos
<code>runs/</code>	métricas, curvas, checkpoints y diagnósticos
<code>scripts/</code>	scripts auxiliares, incluyendo lanzamientos en servidor

La carpeta `data` contiene la lógica específica de cada dataset. Dentro de esta carpeta se encuentra `lab_apparatus.py`, que es el código que implementa la lectura de las imágenes, las anotaciones, los recortes de objetos y las transformaciones necesarias para obtener tensores de entrada. La carpeta `embeddings` contiene las clases que convierten una imagen en una secuencia compatible con la red tensorial. La carpeta `layers` contiene las capas tensoriales propiamente dichas, incluyendo la MPS de referencia, la TTN propia, la versión híbrida y los bloques AIM. Finalmente, `training` y `evaluation` contienen la parte de entrenamiento, métricas y diagnósticos.

Los parámetros de cada experimento se almacenan en archivos YAML dentro de `configs`. Esto permite lanzar entrenamientos distintos cambiando únicamente la configuración; número de épocas, *learning rate*, dimensión de enlace, tipo de *embedding*, tamaño de batch, regularización, *scheduler*, etc. Esta separación fue especialmente útil en la fase experimental, debido a que te permite manipular y comparar parámetros de distintas configuraciones de entrenamiento de manera sencilla y práctica.

4.3 Dataset utilizado

Aunque durante las primeras fases se utilizó el dataset de Oxford Flowers 102 [21], el dataset principal del proyecto es ChemEq25, también referido en el código como *Chemistry Lab Apparatus* [22]. Flowers102 fue útil para comprobar que el pipeline funcionaba con un dataset conocido, descargar datos, generar curvas y verificar que la arquitectura era entrenable. Sin embargo, el objetivo final del TFG es trabajar con imágenes de material de laboratorio, por lo que la metodología experimental terminó centrada en ChemEq25.

ChemEq25 no se usó directamente como una clasificación de imágenes completas. En las imágenes originales pueden aparecer varios objetos, y cada objeto puede pertenecer a una clase distinta. Por tanto, se reformuló el problema como clasificación por instancia; cada anotación del dataset se transforma en un recorte de imagen que contiene un objeto y una etiqueta de clase. Así, el modelo no recibe una escena completa con varios elementos, sino un crop asociado a una única clase.

El dataset contiene 25 clases de material de laboratorio. Tras convertir las anotaciones en recortes individuales, se validaron los siguientes tamaños de split;

Split	Número de crops
Train	4863
Validación	1393
Test	704

Un detalle importante fue que las anotaciones no siempre tenían exactamente el mismo formato. Algunas seguían el formato YOLO habitual de caja delimitadora, mientras que otras contenían más puntos, correspondientes a polígonos de segmentación. Para tratar ambos casos de forma uniforme, cuando una anotación tenía cinco campos se interpretaba como una caja YOLO estándar, y cuando tenía más campos se calculaba la caja delimitadora envolvente del polígono. De esta manera, todas las anotaciones podían convertirse en crops rectangulares.

También se añadieron transformaciones específicas para evitar deformar excesivamente algunos objetos. Al principio, todos los crops se redimensionaban directamente a 224×224 , pero esto podía alterar la relación de aspecto de objetos alargados como pipetas, tubos de ensayo o varillas de vidrio. Por ello se implementó una opción de preprocesamiento que preserva la relación de aspecto y rellena la imagen hasta obtener un cuadrado. El color de relleno se escogió cercano a la media de *ImageNet* para que, tras la normalización, no introdujera bordes artificiales demasiado dominantes.

4.4 El papel del embedding

Una red tensorial no recibe una imagen en formato $[C, H, W]$ de forma directa. Antes es necesario convertir la imagen en una secuencia de vectores locales. En este proyecto esa conversión se denomina *embedding*. De forma general, el embedding transforma una imagen en un tensor de la forma

`[batch, sequence_length, local_dim]`.

Aquí `sequence_length` representa el número de posiciones de entrada que recibe la red tensorial, y `local_dim` es la dimensión del vector asociado a cada sitio. Esta separación es importante porque la red tensorial no trabaja con la geometría original de la imagen, sino con una secuencia.

La primera opción natural era usar una representación muy directa de los píxeles, parecida a la que se emplea en algunos trabajos con MPS sobre datasets sencillos. Sin embargo, esto escala muy mal cuando las imágenes son grandes y en color. Una imagen RGB de 224×224 tendría $224 \cdot 224 \cdot 3 = 150528$ valores si se tratara cada canal de cada píxel como un sitio separado. Aunque esta idea puede ser útil para entender el planteamiento, no era razonable como ruta principal para este proyecto.

Por este motivo se introdujo `PatchEmbedding`. En lugar de tratar cada píxel como un sitio, la imagen se divide en parches. Para imágenes de 224×224 y `patch_size=16`, se

obtiene una rejilla de 14×14 , es decir, 196 posiciones de entrada. Cada patch contiene información local de la imagen y se proyecta mediante una capa lineal a una dimensión `local_dim`. Así se pasa de una imagen a una secuencia compatible con la TTN;

$$[B, 3, 224, 224] \rightarrow [B, 196, \text{local_dim}].$$

Para reducir la pérdida de proximidad espacial al convertir la imagen 2D en una secuencia 1D, se implementó también una ordenación en zigzag. En vez de recorrer todas las filas siempre en el mismo sentido, las filas alternas se recorren en sentido inverso. Esto no conserva toda la geometría 2D, pero evita algunas discontinuidades fuertes entre el final de una fila y el principio de la siguiente.

4.5 Construcción del modelo de TN

Originalmente se consideró utilizar una topología MPS para construir el modelo tensorial. Esta elección era natural, ya que las MPS son una de las arquitecturas de redes tensoriales más estudiadas en tareas de clasificación de imágenes. Sin embargo, gran parte de estos trabajos se han desarrollado sobre datasets relativamente simples, como MNIST o Fashion-MNIST, formados por imágenes en escala de grises y de baja resolución. En MNIST, por ejemplo, cada imagen tiene tamaño 28×28 , por lo que una representación pixel a pixel requiere una secuencia de 784 posiciones de entrada. Aunque este número ya es considerable, sigue siendo manejable y permite realizar las contracciones en un ordenador convencional.

El problema aparece al intentar trasladar esta idea directamente al dataset objetivo de este trabajo. En nuestro caso, las imágenes originales son RGB y tienen una resolución de 640×640 píxeles. Si se construyera una MPS asignando un sitio a cada canal de cada píxel, la longitud de la secuencia sería

$$640 \cdot 640 \cdot 3 = 1\,228\,800.$$

Esto implicaría construir una red con más de un millón de nodos o de posiciones de entrada, lo que hacía inviable utilizar una MPS pixel a pixel como arquitectura principal. Además, en una MPS lineal la información debe propagarse a lo largo de toda la cadena. Para imágenes grandes o secuencias con muchas entradas, esto puede hacer que el modelo sea difícil de entrenar o que requiera una dimensión de enlace demasiado grande.

Por tanto, aunque la MPS fue el punto de partida conceptual por ser la opción más conocida y estudiada, se descartó pronto como solución directa para este problema. Esta limitación motivó el desarrollo de una arquitectura propia de tipo TTN, llamada en el código `CustomTTN`. La idea fue construir una red jerárquica en la que las posiciones de entrada o nodos se agrupan de tres en tres. Cada grupo se contrae mediante un bloque MPS pequeño de tres nodos, y el resultado se usa como entrada del siguiente nivel. El

proceso se repite hasta obtener un único vector raíz.

posiciones de entrada $\rightarrow$ bloques MPS-3 $\rightarrow$ nivel intermedio $\rightarrow \dots \rightarrow$
vector raíz

Cada bloque MPS-3 está formado por tres nodos; izquierdo, central y derecho. El índice libre de salida se situó en el nodo central, de forma que el bloque pudiera contraerse internamente y devolver un vector de dimensión `bond_dim`. Esta elección mantiene una cierta simetría entre los extremos del bloque y permite interpretar la salida como la información que asciende hacia el siguiente nivel de la TTN.

El uso de una TTN reduce la profundidad efectiva de propagación de la información. En una cadena MPS, un sitio puede estar muy lejos de otro en términos de contracciones. En una TTN de aridad 3, el número de niveles crece aproximadamente como $\lceil \log_3(L) \rceil$, donde L es el número de nodos. Por ejemplo, con $L = 196$ posiciones de entrada procedentes de un `PatchEmbedding`, la red necesita pocos niveles jerárquicos en comparación con una cadena lineal. Esta fue una de las razones principales para explorar una topología de árbol.

Un problema práctico aparece cuando el número de posiciones de entrada no es múltiplo de 3. En ese caso, el último bloque de un nivel no tendría tres entradas reales. Para mantener la misma estructura en todos los bloques se añadió un *padding* estructural; nodos constantes no entrenables con tensor $[1, 0, \dots, 0]$. Estos nodos no introducen parámetros nuevos, pero permiten completar los grupos de tres entradas y conservar la arquitectura uniforme.

La TTN propia se implementó usando `TensorKrowch`, una librería que permite construir redes tensoriales sobre PyTorch. Esta decisión tenía sentido porque `TensorKrowch` facilita la creación de nodos, enlaces y contracciones sin tener que escribir manualmente todas las operaciones con `einsum`. Aun así, durante la implementación aparecieron varias dificultades importantes.

La primera fue distinguir entre los nodos estructurales de la red y los nodos de datos que `TensorKrowch` crea al introducir un batch. En la construcción de la red se definen los nodos entrenables, sus ejes y sus conexiones. Después, en el forward, `TensorKrowch` conecta los datos reales a los ejes de entrada. Esta distinción no es solo formal; si los nodos entrenables no se crean correctamente como `ParamNode`, no aparecen como parámetros del modelo y no reciben gradientes.

Otra dificultad importante apareció al contraer niveles sucesivos. Al usar el operador `@`, `TensorKrowch` crea nodos resultantes de una contracción. Sin embargo, los vecinos de ciertos ejes pueden seguir apuntando a los nodos estructurales originales, no necesariamente al nodo resultante que conceptualmente se quiere usar en el siguiente nivel. Por este motivo, en la primera versión de la `CustomTTN` fue necesario seguir explícitamente los nodos compuestos producidos en cada nivel y pasarlos como entradas del nivel siguiente.

Esta parte fue metodológicamente importante porque mostró que implementar una red tensorial no consiste únicamente en que las dimensiones cuadren. También hay que en-

tender cómo la librería representa internamente los nodos, los enlaces y las contracciones. De hecho, varias validaciones sintéticas con longitudes pequeñas, como $L = 9$, $L = 27$ y $L = 196$, fueron necesarias para comprobar que el forward, el backward y los gradientes funcionaban correctamente.

Una vez que la *CustomTTN* funcionaba técnicamente, el siguiente problema fue experimental. Las primeras curvas mostraban que el modelo podía ejecutarse y recibir gradientes, pero su capacidad de aprendizaje y generalización era limitada. Esto llevó a plantear una modificación; introducir transformaciones no lineales entre niveles de la TTN.

La idea era razonable desde el punto de vista del *deep learning*. Si entre niveles solo hay contracciones multilineales, el modelo puede quedar limitado en expresividad o ser difícil de optimizar. Por tanto, se propuso aplicar transformaciones de PyTorch sobre los vectores intermedios, manteniendo siempre la dimensión de enlace;

$$[B, n_blocks, bond_dim] \rightarrow \text{LayerNorm} \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow [B, n_blocks, bond_dim].$$

El problema fue que TensorKrowch no permite reasignar directamente el tensor de un nodo resultante de una contracción. Intentar modificar `node.tensor` después de contraer nodos llevaba a errores, porque esos nodos resultantes no se comportan como tensores ordinarios de PyTorch. Se consideraron varias alternativas; tocar partes internas de TensorKrowch, reescribir toda la arquitectura con `torch.einsum`, o usar directamente una TTN oficial de la librería. Sin embargo, esas opciones se descartaron porque uno de los objetivos del proyecto era crear una TN propia y porque utilizar `torch.einsum` no era lo más eficiente computacionalmente.

La solución fue dividir la red en niveles independientes. Cada nivel de la TTN se encarga de contraer un único nivel de bloques MPS-3 usando TensorKrowch y devuelve un tensor PyTorch ordinario. Entre un nivel y el siguiente se aplican transformaciones estándar de PyTorch. Así nació la *HybridCustomTTN*;

$$\text{TTNLevel} \rightarrow \text{LayerNorm} + \text{Linear} + \text{ReLU} \rightarrow \text{TTNLevel} \rightarrow \dots$$

El nombre *Hybrid* refleja precisamente esta combinación; TensorKrowch se utiliza para las contracciones tensoriales, mientras que PyTorch se utiliza para las transformaciones intermedias y el entrenamiento general del modelo. Esta arquitectura conservó la idea original de una TTN propia, añadió más capacidad expresiva y consiguió una integración más limpia con el flujo de entrenamiento de PyTorch.

4.6 Cambio de embedding

Aunque la *HybridCustomTTN* mejoró el comportamiento respecto a la versión anterior, seguía apareciendo un problema de fondo; la imagen se convertía demasiado pronto en

una secuencia. El `PatchEmbedding` reducía la imagen a una lista de parches y, aunque el orden zigzag ayudaba parcialmente, la estructura 2D original quedaba muy debilitada. Esto es especialmente problemático en imágenes reales, donde la forma, orientación y relación espacial entre partes del objeto son determinantes.

Para atacar este problema se incorporó `AIMPatchEmbedding`, inspirado en los bloques AIM del trabajo *Deep Tree Tensor Networks for Image Recognition*. En lugar de aplanar la imagen casi al principio, primero se procesa como mapa bidimensional mediante un pequeño *front-end* espacial y así mejorar la representación que recibe la TTN.

`imagen → stem CNN → AIM2D → Conv2d 1x1 → flatten + zigzag →
HybridCustomTTN`

El bloque AIM2D tiene dos ramas lineales que se combinan mediante producto elemento a elemento. Si ambas ramas son lineales en la entrada, su producto introduce interacciones bilineales. Esta idea encaja de forma natural con el lenguaje de redes tensoriales, porque introduce interacciones multiplicativas estructuradas antes de entregar la representación a la TTN.

En términos de dimensiones, con imágenes de 224×224 y `patch_size=16`, el *front-end* reduce la imagen a una rejilla 14×14 . Después, una convolución 1×1 proyecta los canales a `local_dim`, y finalmente la rejilla se aplanar;

`[B, 3, 224, 224] → [B, local_dim, 14, 14] → [B, 196, local_dim].`

Este cambio fue importante porque evitó comprimir cada patch directamente a muy pocos números sin haber procesado antes la información espacial. En la configuración final más relevante, se utilizó `local_dim=12` y `bond_dim=12`, de forma que la TTN recibía una representación más rica que en las primeras pruebas.

4.7 Entrenamiento, regularización y métricas de estudio

El entrenamiento se realizó como un problema de clasificación multiclase. El modelo produce *logits* de dimensión igual al número de clases, y la etiqueta de cada crop es un entero entre 0 y 24. La función de pérdida principal fue `CrossEntropyLoss`. El optimizador utilizado fue AdamW, que permite introducir *weight decay* como forma de regularización.

Durante el desarrollo se observó que mirar solo la pérdida de entrenamiento no era suficiente. En algunas fases el modelo aprendía el conjunto de entrenamiento, pero no generalizaba bien al conjunto de validación. Por eso se registraron varias métricas complementarias;

- **Train loss y eval loss**; permiten ver si el modelo está aprendiendo y si aparece separación entre entrenamiento y validación.
- **Accuracy top-1**; mide si la clase más probable coincide con la etiqueta correcta.

- **Accuracy top-5**; mide si la clase correcta aparece entre las cinco clases más probables. Es útil cuando hay clases visualmente parecidas.
- **Macro F1**; resume el comportamiento medio por clase, dando peso similar a todas las clases.
- **Confianza media en aciertos y errores**; ayuda a detectar si el modelo falla con demasiada seguridad.

Además, se añadieron diagnósticos por clase; matriz de confusión, métricas por clase, distribución de predicciones y lista de confusiones más frecuentes. Esto permitió identificar que los errores no eran aleatorios, sino que se concentraban en objetos visualmente parecidos o geoméricamente delicados, como *Measuring Cylinder*, *Volumetric Flask*, *Separating Funnel*, *Test Tube*, *Pipette* o las distintas variantes de *Round Bottom Flask*.

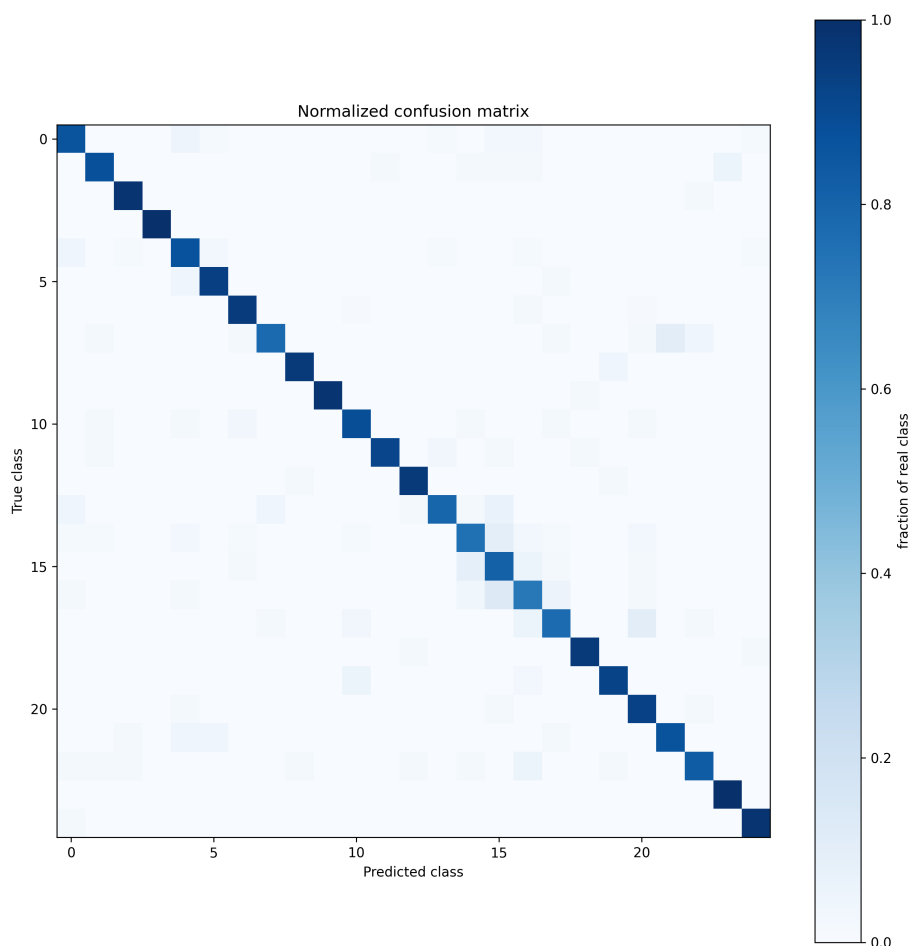


Figura 4.2: Matriz de confusión obtenida experimentalmente utilizando el modelo *Hybrid-CustomTTN*

A partir de estos diagnósticos se fueron introduciendo mejoras en el entrenamiento. El

learning rate se redujo respecto a las primeras pruebas, porque valores demasiado altos producían trayectorias de optimización menos estables. Después se añadió un *scheduler ReduceLROnPlateau*, que reduce el *learning rate* cuando la pérdida de validación deja de mejorar. También se implementó *warmup*, primero lineal y después coseno, para que el entrenamiento no empezara con pasos demasiado agresivos desde la primera época.

Otra mejora fue la regularización explícita *L2*, añadida a la función de pérdida además del *weight decay*. Finalmente se incorporó *label smoothing*, que suaviza ligeramente la confianza durante el entrenamiento. Su objetivo no era solo mejorar la precisión, sino reducir la sobreconfianza del modelo cuando se equivoca. En las pruebas finales, `label_smoothing=0.05` produjo una mejora pequeña en *accuracy*, pero una mejora más clara en calibración.

4.8 Evolución experimental del proyecto

La evolución del proyecto puede resumirse como una cadena de problemas y respuestas. Primero se ideó una MPS y un pipeline de entrenamiento funcional. Después se desarrolló una TTN propia para reducir las limitaciones de una cadena lineal y explorar una topología jerárquica. Más adelante, al ver que la TTN era difícil de enriquecer con no linealidades, se refactorizó en una *HybridCustomTTN* por niveles. Finalmente, al observar que la representación secuencial perdía demasiada información espacial, se incorporó *AIMPatchEmbedding*.

Esta evolución también afectó al dataset. *Flowers102* sirvió como banco de pruebas inicial, pero el trabajo experimental se trasladó a *ChemEq25*, que era el dataset más alineado con el objetivo del TFG. La adaptación a *ChemEq25* obligó a implementar lectura de anotaciones, generación de crops, auditorías de datos y transformaciones específicas para preservar mejor la forma de los objetos.

Los primeros entrenamientos serios sobre *ChemEq25* con AIM y *HybridCustomTTN* ya alcanzaron resultados alrededor del 70–75 % de *accuracy* en validación. Posteriormente, al ajustar el *learning rate*, añadir *warmup*, preservar la relación de aspecto, usar regularización y aplicar *label smoothing*, la mejor configuración observada alcanzó aproximadamente un 89 % de *accuracy* y un macro F1 también cercano a 0.89. Estos resultados concretos se analizarán con más detalle en el capítulo de resultados y discusión; aquí lo importante es entender que cada mejora nació de un diagnóstico experimental previo.

4.9 Ejecución en servidores y registro experimental

Los entrenamientos más costosos se ejecutaron en los servidores Lorca de la universidad. Esto formó parte de la metodología real del proyecto, porque permitió lanzar entrenamientos largos, usar GPU, guardar checkpoints y recuperar métricas para analizarlas posteriormente. El trabajo en servidor también obligó a cuidar aspectos prácticos como el entorno de Python, las dependencias, los scripts de lanzamiento y los logs de ejecución.

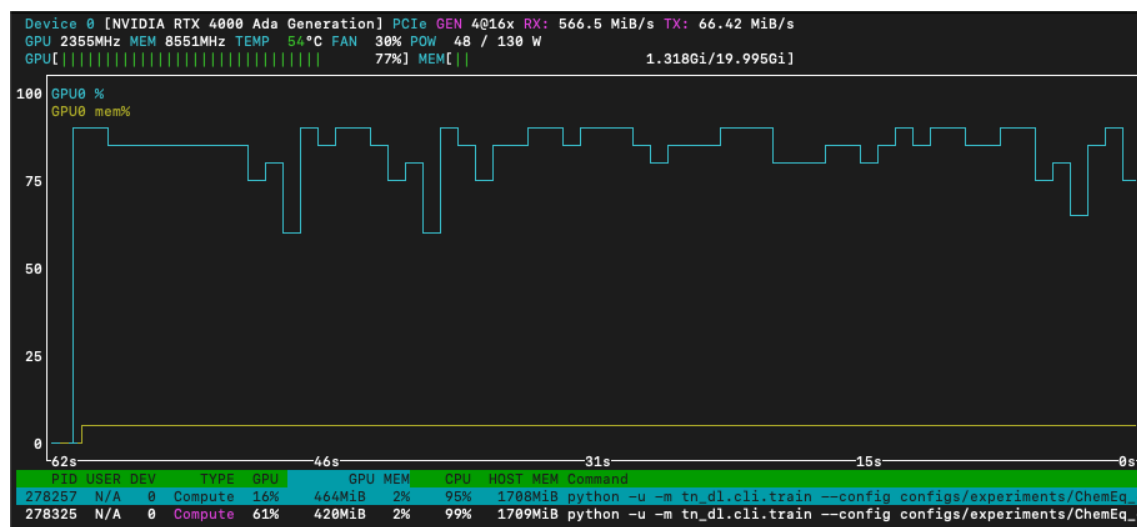


Figura 4.3: Captura de pantalla de una GPU ADA 4000 del servidor Lorca de la UEM ejecutando dos procesos de entrenamientos diferentes en paralelo.

Cada ejecución genera una carpeta dentro de `runs/`. En ella se guardan los archivos necesarios para reconstruir el experimento; configuración usada, métricas por época, curvas de pérdida, checkpoints, diagnósticos y, cuando procede, matrices de confusión. Esta trazabilidad fue esencial para comparar configuraciones de forma razonable. Sin este registro, habría sido muy difícil saber si una mejora se debía realmente a la arquitectura, al preprocesamiento, al *learning rate*, a la seed o a otro cambio del entrenamiento.

Capítulo 5. RESULTADOS

Capítulo 6. DISCUSIÓN

Capítulo 7. CONCLUSIONES

7.1 Conclusiones del trabajo

Breve descripción objetiva del resultado en relación al objetivo general de tu TFG.

7.2 Conclusiones personales

Describe tus impresiones y experiencia personal durante el desarrollo del TFG, o destacar la importancia que tiene el tema para ti, lo que has aprendido, o la trascendencia que ha tenido para ti o para otros este trabajo.

REFERENCIAS

- [1] Firefly meta-heuristic algorithm applied to artificial neural network pre-training. Scientific Figure on ResearchGate.
- [2] Fred Van Veen and Stefan Leijnen. The neural network zoo. <https://www.asimovinstitute.org/neural-network-zoo>, 2019. Asimov Institute. Accessed: 2026-03-19.
- [3] Alejandro Mata Ali, Iñigo Perez Delgado, and Aitor Moreno Fdez. de Leceta. Quantum-inspired Techniques in Tensor Networks for Industrial Contexts. *arXiv e-prints*, page arXiv:2404.11277, April 2024.
- [4] Maolin Wang, Yu Pan, Zenglin Xu, Guangxi Li, Xiangli Yang, Danilo Mandic, and Andrzej Cichocki. Tensor networks meet neural networks: A survey and future perspectives. *arXiv preprint arXiv:2302.09019*, 2025. Version 3, revised 17 Mar 2025.
- [5] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P Mandic. Tensor networks for dimensionality reduction and large-scale optimization part 1 low-rank tensor decompositions. *Foundations and Trends in Machine Learning*, 9(4-5):249–429, 2016.
- [6] E. Miles Stoudenmire and David J. Schwab. Supervised learning with quantum-inspired tensor networks, 2017.
- [7] Song Cheng, Lei Wang, and Pan Zhang. Supervised learning with projected entangled pair states. *Physical Review B*, 103(12):125117, 2021.
- [8] Norbert Schuch, Michael M. Wolf, Frank Verstraete, and J. Ignacio Cirac. Computational complexity of projected entangled pair states. *Physical Review Letters*, 98(14):140506, 2007.
- [9] G. Evenbly and G. Vidal. Tensor Network States and Geometry. *Journal of Statistical Physics*, 145(4):891–918, November 2011.
- [10] Chang Nie, Junfang Chen, and Yajie Chen. Deep tree tensor networks for image recognition, 2025. NeurIPS 2025 poster; arXiv:2502.09928.
- [11] José Ramón Pareja Monturiol, David Pérez-García, and Alejandro Pozas-Kerstjens. Tensorkrowch: Smooth integration of tensor networks in machine learning. *Quantum*, 8:1364, 2024.
- [12] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.

- [13] Roger Penrose et al. Applications of negative dimensional tensors. *Combinatorial mathematics and its applications*, 1(221-244):3, 1971.
- [14] Jacob Biamonte and Ville Bergholm. Tensor networks in a nutshell, 2017.
- [15] Ulrich Schollwoeck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics - ANN PHYS N Y*, 326, 08 2010.
- [16] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.
- [17] Ivan Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33:2295–2317, 09 2011.
- [18] G. Vidal. Entanglement renormalization. *Phys. Rev. Lett.*, 99:220405, Nov 2007.
- [19] F. Verstraete and J. I. Cirac. Renormalization algorithms for quantum-many body systems in two and higher dimensions. *arXiv preprint cond-mat/0407066*, 2004.
- [20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [21] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics and Image Processing*, Dec 2008.
- [22] Md Sakhawat Hossain et al. Real-world chemistry lab image dataset for equipment recognition across 25 apparatus categories. *Scientific Data*, 12(1747), 2025.

ANEXOS

Sirven para incluir documentación complementaria (planos, circuitos, código, ficheros de configuración, especificaciones técnicas y hojas de características, fichas explicativas, resultado de encuestas, reglamentación y normativas requeridas, etc.).

[PÁGINA INTENCIONADAMENTE EN BLANCO]